Le microservice Notification Service

Projet CampusFind, ESPRIT - Document de conception architecturale et fonctionnelle

Le microservice Notification Service informe les étudiants des événements provenant des six microservices métier. Il reçoit les événements via RabbitMQ, enregistre les notifications dans MongoDB et les met à disposition du frontend. Il ne gère pas la messagerie privée et ne stocke pas les données métier des autres services.

1. Conception du microservice : diagramme de classes

Le service possède quatre documents MongoDB : Notification, PreferenceNotification, TentativeEnvoi et EvenementTraite. Les énumérations sont placées à côté des classes. Le champ id représente un ObjectId MongoDB exposé en String dans Node.js.

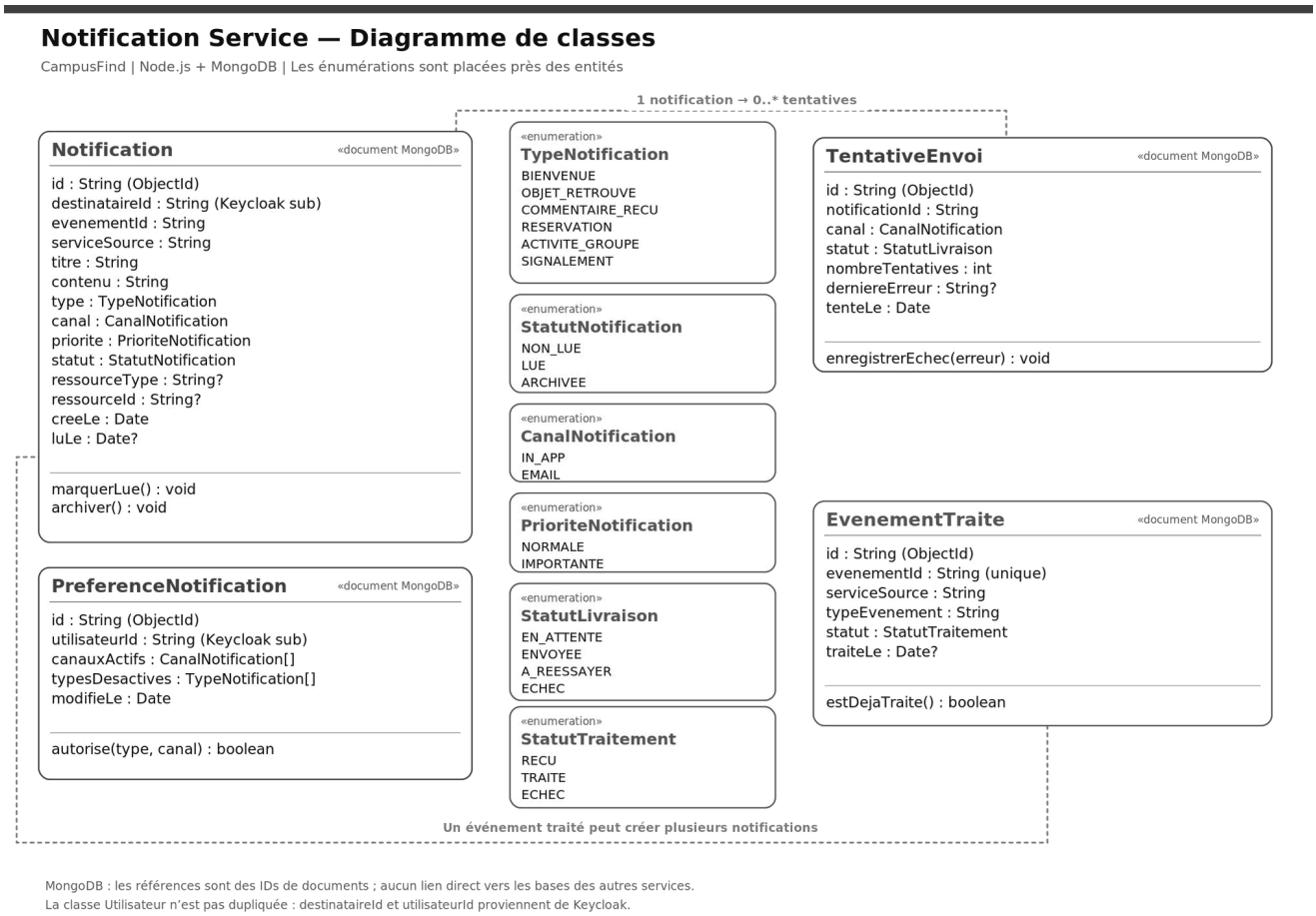


Figure 1 - Diagramme de classes du Notification Service et ses énumérations.

Les identifiants des étudiants proviennent du claim sub de Keycloak. Chaque document possède un seul champ id ; aucune base de données des autres microservices n'est partagée.

2. Interactions avec les autres microservices

Les six microservices métier publient des événements dans RabbitMQ. Notification Service les consomme de façon asynchrone. Le frontend consulte ses notifications par une API REST synchrone via l'API Gateway.

Microservice source	Événement proposé	Destinataire
Lost & Found	ObjetRetrouve	Auteur de l'annonce correspondante
User Management	AccountCreated	Étudiant concerné (bienvenue)
Student Expression	PostCommented	Auteur de la publication
Room Reservation	ReservationConfirmed / ReservationCancelled	Réservataire de la salle
Study Community	NewGroupActivity / GroupSessionCancelled	Membres concernés du groupe
Campus Issues	IssueStatusChanged	Auteur du signalement

2.1 Communication asynchrone : RabbitMQ

1. Le microservice métier enregistre l'action dans sa propre base (exemple : confirmation d'une salle).

2. Il publie un événement contenant eventId, sourceService, eventType, recipientIds et les informations utiles.

3. RabbitMQ reçoit l'événement sur un exchange de type topic, puis le dirige vers la file notifications.queue.

4. Le worker Node.js lit le message, vérifie les préférences et les doublons, puis enregistre les notifications dans MongoDB.

5. Après sauvegarde réussie, il confirme le traitement (ack). En cas d'échec, il réessaie ou utilise une DLQ.

Exemple : Study Community publie NewGroupActivity lorsqu'une séance est confirmée. Notification Service enregistre une notification pour les membres concernés. Room Reservation peut aussi publier ReservationConfirmed, avec des destinataires définis pour éviter une double alerte.

2.2 Communication synchrone : consultation REST

L'étudiant se connecte avec Keycloak. Angular appelle GET /api/notifications/me via l'API Gateway, avec son JWT. Le serveur Express vérifie le jeton et renvoie uniquement les notifications de cet étudiant (200 OK). Un PATCH permet de marquer une notification comme lue.

2.3 Vue d'ensemble des interactions (explication sans schéma)

Le tableau suivant remplace le diagramme d'interaction. Il présente le chemin des demandes et des événements, ainsi que les technologies utilisées.

Émetteur	Destinataire	Mode / technologie	Action réalisée
6 services métier	RabbitMQ	Asynchrone - événement	Publier les événements utiles aux étudiants.
RabbitMQ	Notification Service	Asynchrone - queue	Transmettre les événements à traiter au worker Node.js.
Notification Service	MongoDB	Accès aux données	Créer la notification, enregistrer les préférences et éviter les doublons.
Frontend Angular	API Gateway	Synchrone - REST + JWT	Demander la liste des notifications de l'étudiant.
API Gateway	Notification Service	Synchrone - REST	Acheminer la requête et retourner la réponse HTTP.
Keycloak	Notification Service	Sécurité - JWT/JWKS	Permettre la vérification de l'identité de l'étudiant.

2.4 Contrat minimal d'un événement

Tous les microservices doivent se mettre d'accord sur les champs suivants avant de développer la communication RabbitMQ :

Champ	Signification
eventId	Identifiant unique permettant de reconnaître un même événement.
sourceService	Nom du microservice qui a publié l'événement.
eventType	Type d'événement (ex. NewGroupActivity).
occurredAt	Date et heure de l'événement.
recipientIds	Liste des identifiants Keycloak des destinataires.
payload	Données utiles à l'affichage de la notification, sans secret ni mot de passe.

2.5 Gestion des erreurs et des pannes

Situation	Comportement prévu
Un service métier est indisponible	Les autres services continuent à publier normalement.
Notification Service est arrêté	Les messages déjà acceptés restent dans une file durable si RabbitMQ est correctement configuré.
Le même événement arrive deux fois	Dédupliquer avec le couple sourceService + eventId (index unique MongoDB).
Erreur pendant le traitement	Effectuer des tentatives contrôlées puis orienter vers une file d'erreurs (DLQ).
MongoDB indisponible	Ne pas acquitter le message avant la persistance ; le traiter lors d'une nouvelle tentative.

3. Contrats d’API - Notification Service

Ces endpoints REST sont protégés par JWT. L’utilisateur peut consulter et modifier uniquement ses propres notifications et préférences.

Méthode	Endpoint	Fonction	Réponse
GET	/api/notifications/me	Lister mes notifications	200
GET	/api/notifications/me/unread-count	Compter les notifications non lues	200
PATCH	/api/notifications/{id}/read	Marquer une notification comme lue	200 / 404
PATCH	/api/notifications/me/read-all	Tout marquer comme lu	200
GET	/api/notifications/preferences/me	Consulter mes préférences	200
PUT	/api/notifications/preferences/me	Modifier mes préférences	200 / 400

4. Interactions avec l’infrastructure

Composant	Rôle
Node.js + Express	Expose l’API REST et exécute le worker RabbitMQ dans le même microservice.
RabbitMQ	Broker : exchanges, files durables et file d’erreurs (DLQ).
Keycloak	Fournit le JWT ; Node.js vérifie sa signature avec JWKS.
API Gateway	Route /api/notifications/** vers le service Node.js.
Eureka	Découverte des services Spring ; Node.js ne s’enregistre pas automatiquement.
Config Server	Principalement pour Spring ; Node.js peut utiliser des variables d’environnement.
Docker Compose	Lance Node.js, MongoDB, RabbitMQ et les autres composants.

5. Choix de MongoDB

MongoDB convient aux notifications car leur contenu varie selon les événements (réservation, commentaire, objet retrouvé, signalement). La base notifications_db comporte quatre collections : notifications, notification_preferences, delivery_attempts et processed_events.

Index prévus : (destinataireId, creeLe), (destinataireId, statut), unicité (sourceService, evenementId) dans processed_events et unicité (serviceSource, evenementId, destinataireId) dans notifications.

6. Énumérations

Énumération	Valeurs
TypeNotification	BIENVENUE, OBJET_RETROUVE, COMMENTAIRE_RECU, RESERVATION, ACTIVITE_GROUPE, SIGNALEMENT
CanalNotification	IN_APP, EMAIL (optionnel)
StatutNotification	NON_LUE, LUE, ARCHIVEE
PrioriteNotification	NORMALE, IMPORTANTE
StatutLivraison	EN_ATTENTE, ENVOYEE, A_REESSAYER, ECHEC
StatutTraitement	RECU, TRAITE, ECHEC

7. Règles de fonctionnement

Un étudiant voit uniquement ses notifications. Une notification n’est créée qu’une fois par événement et par destinataire. Les préférences peuvent désactiver certaines catégories. Le service reste indépendant des données métier et ne gère pas les conversations.